

면접 질문 전집: STAR 기법 답변 (nano-vllm)

사용 설명: 각 문제는 S-T-A-R 구조를 채택합니다; **Action**에는 가능한 한 `nano-vllm-main` 하에 위치 추적 가능한 소스 코드 단서(파일명과 로직)를 제시합니다.

주의: 기본값 `kvcache_block_size`는 저장소 `config.py`를 기준으로 합니다(예제 저장소는 256); 만약 로컬에서 16으로 변경했다면, 아래의 '모듈로' 관련 결론은 여러분의 `block_size`로 동기화하여 대체하세요.

1. 프로젝트 전반적 이해 (8문제)

Q1: nano-vllm 프로젝트를 소개해 주세요.

Situation: 대규모 모델 추론 공학은 경로가 길어, 초대형 프레임워크 소스 코드를 직접 읽는 비용이 높습니다; '모듈이 완전하고, 코드 양이 통제 가능한' 학습 경로가 필요합니다.

Task: 제한된 시간 내에 '스케줄링-메모리-연산-통신'에 대한 일관된 인지를 구축하고, 산업계 vLLM 계열 아키텍처와 대조할 수 있도록 합니다.

Action: nano-vllm은 약 천 줄 규모의 파이썬으로 구성된 교습 목적의 추론 엔진으로, 진입점은 `LLMEngine`입니다: 요청이 `Scheduler` 큐에 들어가고, `step()`을 통해 `ModelRunner`가 순전파 및 샘플링을 실행하도록 구동합니다. 핵심 구성: `BlockManager`의 페이징 KV, `Scheduler`의 연속 배치 처리, `Attention` 내 FlashAttention 이중 경로 + Triton KV 쓰기, `ModelRunner` 내 KV 텐서 할당 및 선택적 CUDA Graph.

Result: 프로젝트를 「아키텍처는 vLLM 사상에 정렬되고, 구현은 극도로 간소화된」 학습 매개체로 포지셔닝할 수 있습니다. 답변 시 포함되지 않은 산업적 특성(예: 완전한 양자화, 추측적 디코딩 등)을 능동적으로 설명하여 경계 감각을 보여줍니다.

소스 코드 앵커:

```
def step(self):
    seqs, is_prefill = self.scheduler.schedule()
    token_ids = self.model_runner.call("run", seqs, is_prefill)
    self.scheduler.postprocess(seqs, token_ids)
    outputs = [(seq.seq_id, seq.completion_token_ids) for seq in seqs if seq.is_finished]
    num_tokens = sum(len(seq) for seq in seqs) if is_prefill else -len(seqs)
    return outputs, num_tokens
```

Q2: nano-vllm의 전체 아키텍처는 어떻게 구성되어 있나요?

Situation: 면접관은 파일명 나열이 아닌 '계층화'된 설명을 듣고 싶어 합니다.

Task: 하나의 주요 흐름으로 모듈을 연결합니다: 누가 상태를 보유하고, 누가 결정을 내리며, 누가 계산을 하는가.

Action:

- **API / 엔진 계층:** `LLMEngine`이 `Tokenizer`, `Scheduler`, `ModelRunner`를 조합합니다(멀티프로세스 시 rank0이 스케줄링, 다른 rank는 SharedMemory를 통해 동기화 호출).
- **스케줄링 계층:** `Scheduler`가 `waiting`과 `running`을 유지하며, `schedule()`이 이번 단계에 참여할 `Sequence` 리스트 및 Prefill 여부를 산출합니다.
- **메모리 계층:** `BlockManager`가 물리 블록 풀과 각 시퀀스의 `block_table`을 관리하며, 프리픽스 해시와 연계합니다.
- **실행 계층:** `ModelRunner`가 `[2, L, num_blocks, block_size, kv_heads, head_dim]` 형태의 KV를 할당하고, `slot_mapping`, `cu_seqlens`, `block_tables` 등의 컨텍스트를 구성하여 모델과 `Sampler`를 호출합니다.

Result: 도식으로 표현하면 '요청 → Sequence → 스케줄링 → 컨텍스트 텐서 → Attention/FFN → logits → 샘플링 → postprocess'.

소스 코드 앵커:

```
self.scheduler = Scheduler(config)
```

```
def allocate_kv_cache(self):
    ...
    self.kv_cache = torch.empty(2, hf_config.num_hidden_layers, config.num_kvcache_blocks,
```

Q3: 하나의 요청이 입력부터 출력까지 거치는 단계는 무엇인가요?

Situation: 한 번의 forward가 아닌 **비동기 큐 + 단계별 decode**를 이해하는지 검증하는 데 사용됩니다.

Task: `add_request` 이후 발생하는 일을 단계별로 설명합니다.

Action:

1. `add_request` 가 프롬프트를 `token_ids` 로 인코딩하고, `Sequence` 를 구성하여, `scheduler.add` 로 `waiting` 에 넣습니다.
2. 매 라운드 `step()` : `schedule()` 이 이번 배치의 시퀀스를 선택합니다; `Prefill` 진행 시, `prepare_prefill` 로 여러 시퀀스를 패킹합니다; `Decode` 진행 시, `prepare_decode` 는 시퀀스당 1개의 새 토큰만 처리합니다.
3. `ModelRunner.run` 내 `run_model` 이 logits를 계산하고, rank0에서 `Sampler` 로 샘플링하여 토큰 id를 얻습니다.
4. `postprocess` 가 토큰을 `Sequence` 에 추가하고, EOS 여부/`max_tokens` 도달 여부를 판단하여, 종료 시 `deallocate` 합니다.

Result: `Prefill`은 여러 토큰을 한 번에 처리할 수 있고, `Decode`는 단계당 시퀀스당 1 토큰임을 vLLM과 일관되게 강조할 수 있습니다.

소스 코드 앵커:

```
def add_request(self, prompt: str | list[int], sampling_params: SamplingParams):
    ...
    seq = Sequence(prompt, sampling_params)
    self.scheduler.add(seq)
```

```
def postprocess(self, seqs: list[Sequence], token_ids: list[int]) -> list[bool]:
    for seq, token_id in zip(seqs, token_ids):
        seq.append_token(token_id)
        if (not seq.ignore_eos and token_id == self.eos) or seq.num_completion_tokens == s
            seq.status = SequenceStatus.FINISHED
            self.block_manager.deallocate(seq)
            self.running.remove(seq)
```

Q4: nano-vllm과 vLLM의 차이점은 무엇인가요?

Situation: '비슷하다' 혹은 '많이 다르다'는 두 극단적인 답변을 피합니다.

Task: 기능 범위, 엔지니어링, 생태계 측면에서 비교합니다.

Action: nano-vllm은 코드 양이 극히 적고, 구조가 명확하며, `PagedAttention` 사상, 연속 배치 처리, `FlashAttention`, `TP`, `CUDA Graph`, `Triton KV` 쓰기 등 주요 골격을 커버합니다. vLLM은 기능이 더 완전하며(더 많은 모델, 양자화, `PagedAttention` 주변 전략, 프로덕션급 내결함성 및 스케줄링 변형 등), 코드 양과 의존성이 더 무겁습니다.

Result: "nano-vllm을 학습하여 골격을 세우고, 다시 vLLM을 읽어 공학적 디테일을 보완한다"고 표현하여-성장 경로를 보여줍니다.

Q5: nano-vllm을 학습하기로 선택한 이유는 무엇인가요?

Situation: 면접관이 동기와 계획을 탐색합니다.

Task: 막연한 흥미가 아닌 검증 가능한 학습 계획을 제시합니다.

Action: vLLM 전체 소스 코드 읽기는 주기가 길습니다. nano-vllm은 짧은 주기 내에 '스케줄링 → KV → 어텐션 → 샘플링' 폐쇄 루프를 실행할 수 있으며, 논문/블로그의 개념과 일대일 대응되어 실험과 노트 작성에 용이합니다.

Result: 이미 완료한 산출물(벤치마크, 주식 포크, 마인드맵 등)을 보충할 수 있습니다.

Q6: 이 프로젝트에서 무엇을 배웠나요?

Situation: 행동 + 기술 종합 문제.

Task: 전이 가능한 능력을 요약합니다: 코드 읽기, 실험하기, 명확히 설명하기.

Action: 세 가지 범주의 수확을 예시로 듭니다:

- **시스템:** 두 단계 스케줄링과 메모리 부족 시 선점의 트레이드오프;
- **연산자:** FlashAttention의 prefill/decode API 상의 차이;
- **엔지니어링:** CUDA Graph의 전제 조건과 `enforce_eager` 스위치.

Result: "추론 엔진의 2차 개발 / 문제 해결의 기초를 감당할 수 있다"는 결론으로 이어집니다.

Q7: nano-vllm을 개선한다면, 어떻게 하시겠습니까?

Situation: 개방형 질문, 우선순위와 아키텍처 감각을 평가합니다.

Task: 투입 대비 산출 순으로 정렬된 로드맵을 제시합니다.

Action:

- 단기: **Chunked Prefill**로 긴 프롬프트가 decode를 막는 현상 완화; 또는 모니터링과 단위 테스트 보완.
- 중기: **가중치/KV 양자화**, **HTTP API**, 스트리밍 출력.
- 장기: **추측적 디코딩**, **PD 분리**(분산 및 KV 전송 필요).

각 항목마다 수정이 필요한 모듈(예: `scheduler.py`, `model_runner.py`)을 설명합니다.

Result: "어려운 점은 스케줄링과 상태 일관성에 있으며, 단순히 함수 하나를 수정하는 것이 아니다"를 알고 있음을 보여줍니다.

Q8: nano-vllm의 성능 병목은 어디에 있을 수 있나요?

Situation: Prefill과 Decode를 구분해야 합니다.

Task: 구현을 결합하여 검증 가능한 병목 가설을 제시합니다.

Action:

- **Prefill:** 시퀀스가 길 때 어텐션 계산량 ($O(n^2)$)이 주도; Graph는 보통 사용되지 않음(`run_model` 조건 참조).
- **Decode:** 단계별 연산자가 많고, 런칭이 빈번; 프로젝트에서 **CUDA Graph**로 완화; 여전히 **메모리 대역폭**과 **TP 통신**의 제한을 받을 수 있음.
- **스케줄링:** 극단적 부하 하에서 선점 및 prefill 재계산이 꼬리 지연을 증가시킬 수 있음.

Result: 답변 시 **Profiler / Nsight**로 검증할 것임을 언급하며, 단일 병목을 단언하지 않습니다.

소스 코드 앵커:

```
def run_model(self, input_ids: torch.Tensor, positions: torch.Tensor, is_prefill: bool):
    if is_prefill or self.enforce_eager or input_ids.size(0) > 512:
        return self.model.compute_logits(self.model(input_ids, positions))
```

2. KV Cache 및 PagedAttention (8문제)

Q9: KV Cache의 작동 원리를 설명해 주세요.

Situation: 자기회귀 생성은 각 단계마다 과거 토큰의 K/V에 의존하며, 반복 계산 비용이 극도로 높습니다.

Task: '무엇을 캐시하는지, 언제 기록하는지, 어떻게 후속 단계에서 재사용하는지'를 설명합니다.

Action: 각 레이어는 먼저 현재 단계의 K, V를 계산합니다; 과거 단계의 K/V는 캐시에 저장됩니다; 후속 단계에서는 **새로운 query**와 **전체 과거 K/V**에 대해서만 어텐션을 수행합니다. nano-vllm은 캐시를 대형 `kv_cache` 텐서에 배치하

고, `slot_mapping` 을 통해 기록 위치를 지정합니다; Decode 시 `flash_attn_with_kvcache` 를 사용하여 캐시를 직접 읽습니다.

Result: '캐시 없이 매 단계 전체를 재계산'하는 경우와 '캐시로 점진적 갱신만 하는' 경우의 복잡도 차이를 비교할 수 있습니다.

소스 코드 앵커:

```
def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor):
    context = get_context()
    k_cache, v_cache = self.k_cache, self.v_cache
    if k_cache.numel() and v_cache.numel():
        store_kvcache(k, v, k_cache, v_cache, context.slot_mapping)
    if context.is_prefill:
        ...
    else: # decode
        o = flash_attn_with_kvcache(q.unsqueeze(1), k_cache, v_cache,
                                    cache_seqlens=context.context_lens, block_table=context
                                    softmax_scale=self.scale, causal=True)
```

Q10: PagedAttention은 메모리 파편화를 어떻게 해결하나요?

Situation: 최대 길이로 연속 할당하면, 짧은 시퀀스가 꼬리 공간을 낭비하고, 동시성이 많을 때 '총 여유 공간은 충분하지만 큰 연속 블록을 할당할 수 없는' 문제가 두드러집니다.

Task: 페이징이 어떻게 고정 크기 블록 할당으로 변경하는지 설명합니다.

Action: KV는 물리적으로 통일된 풀에 배치되며, 각 시퀀스는 `block_table` 로 논리 블록에서 물리 블록 ID로의 매핑을 기록합니다; 할당/회수는 블록 단위로 이루어지며, 내부 파편화는 **마지막 불완전 블록**으로 제한됩니다.

Result: '연속 할당'과 '블록 할당'의 파편화 원인 차이를 비교하고, `free_block_ids` 를 가리킬 수 있습니다.

소스 코드 앵커:

```
class BlockManager:

    def __init__(self, num_blocks: int, block_size: int):
        self.block_size = block_size
        self.blocks: list[Block] = [Block(i) for i in range(num_blocks)]
        self.hash_to_block_id: dict[int, int] = dict()
        self.free_block_ids: deque[int] = deque(range(num_blocks))
```

Q11: Block Table의 데이터 구조와 유지 관리 로직은 무엇인가요?

Situation: 면접관이 논리 블록과 물리 블록의 매핑을 탐색합니다.

Task: 누가 `table`을 보유하는지, 언제 추가하는지, `Sequence` 길이와 어떤 관계인지 설명합니다.

Action: `Sequence.block_table` 은 물리 블록 ID 리스트입니다; `allocate` 가 처음으로 전체 프롬프트에 대해 할당합니다; `may_append` 가 블록이 가득 찼을 때 새로운 물리 블록 ID를 추가합니다; `len(seq)`, `num_blocks` 속성과 연동됩니다.

Result: '시퀀스 길이가 0부터 증가할 때, 언제 새 블록이 필요한지'를—`can_append` 와 연동하여—손으로 계산할 수 있습니다.

소스 코드 앵커:

```
def can_append(self, seq: Sequence) -> bool:
    return len(self.free_block_ids) >= (len(seq) % self.block_size == 1)

def may_append(self, seq: Sequence):
    block_table = seq.block_table
```

```
last_block = self.blocks[block_table[-1]]
if len(seq) % self.block_size == 1:
    ...
    block_table.append(block_id)
```

Q12: 프리픽스 캐시는 어떻게 구현되어 있나요?

Situation: 여러 요청이 동일한 프롬프트 프리픽스를 공유할 때, Prefill을 반복 계산하면 연산 능력이 낭비됩니다.

Task: 해시 키, 적중 조건 및 참조 카운팅을 설명합니다.

Action: `compute_hash` 가 프리픽스 해시 + 현재 블록 토큰 바이트를 결합합니다; 가득 찬 블록이 `hash_to_block_id` 에 기록됩니다; `allocate` 중 해시가 적중하고 `token_ids` 가 일치하면 재사용하고 `ref_count` 를 증가시킵니다.

Result: '왜 `token_ids` 2차 검증이 필요한지'-해시 충돌로 인한 잘못된 재사용 방지-를 설명할 수 있습니다.

소스 코드 앵커:

```
@classmethod
def compute_hash(cls, token_ids: list[int], prefix: int = -1):
    h = xxhash.xxh64()
    if prefix != -1:
        h.update(prefix.to_bytes(8, "little"))
    h.update(np.array(token_ids).tobytes())
    return h.intdigest()
```

Q13: KV Cache 블록이 몇 개 필요한지 어떻게 계산하나요?

Situation: 스케줄링 및 OOM 분석에서 자주 묻습니다.

Task: 시퀀스 길이와 `block_size` 로부터 블록 수를 유도하고, 논리 블록과 물리 풀 크기를 구분합니다.

Action: 단일 시퀀스는 약 $(\lceil \text{len} / \text{block_size} \rceil)$ 개의 논리 블록이 필요합니다; 물리 풀 블록 수는 `allocate_kv_cache` 에서 메모리 예산을 단일 블록 바이트 수로 나누어 구합니다.

Result: `block_bytes` 와 레이어 수, 헤드 차원, `block_size` 의 관계를 쓸 수 있어야 합니다(`allocate_kv_cache` 참조).

소스 코드 앵커:

```
num_kv_heads = hf_config.num_key_value_heads // self.world_size
head_dim = getattr(hf_config, "head_dim", hf_config.hidden_size // hf_config.num_attn_heads)
block_bytes = 2 * hf_config.num_hidden_layers * self.block_size * num_kv_heads * head_dim
config.num_kvcache_blocks = int((total * config.gpu_memory_utilization - used - peak +
```

Q14: deallocate 시 왜 block_table을 역방향으로 순회하나요?

Situation: 참조 카운팅과 프리픽스 공유 시나리오를 평가합니다.

Task: 해제 순서가 정확성에 영향을 미치는지 설명합니다; 여기서는 주로 블록별로 `ref_count -= 1` 을 수행합니다.

Action: 역방향 순회는 체인/공유 구현에서 자주 '시퀀스 꼬리부터 후퇴'하는 것과 일관됩니다; 본 구현에서는 각 `block_id` 가 독립적으로 참조를 감소시키므로, 순서는 수치에 영향을 주지 않지만 코드 관행상 뒤에서 앞으로 해제하는 것이 시퀀스 꼬리 블록이 먼저 참조가 없어지는 것에 더 가깝습니다(구체적인 캐시 전략과 함께 기억할 수 있습니다).

Result: '블록별 ref가 독립적으로 감소하고, 0이 되면 `free_block_ids` 에 반환된다'고 답변하면 점수를 얻을 수 있습니다; 만약 공유를 추궁한다면, '프리픽스 블록이 여러 시퀀스에 의해 참조될 수 있다'고 연결합니다.

소스 코드 앵커:

```
def deallocate(self, seq: Sequence):
    for block_id in reversed(seq.block_table):
        block = self.blocks[block_id]
```

```

        block.ref_count -= 1
        if block.ref_count == 0:
            self._deallocate_block(block_id)
seq.num_cached_tokens = 0
seq.block_table.clear()

```

Q15: can_append의 판단 로직은 무엇인가요? (len(seq) % block_size == 1일 때 새 블록이 필요함)

Situation: Decode 시 단계마다 길이가 +1 증가하며, 언제 블록을 넘는지 핵심 디테일입니다.

Task: 불리언 표현식 `len(self.free_block_ids) >= (len(seq) % block_size == 1)` 을 설명합니다.

Action: 파이썬에서 `len(seq) % block_size == 1` 이 True일 때, 다음 append는 새 길이를 '새 블록 시작점'으로 만들며-**사전에** 블록 하나를 더 점유해야 합니다; 따라서 빈 블록 수가 최소 1이어야 합니다. False이면, 여전히 현재 블록 내에 머무르므로 새 블록이 필요하지 않습니다.

Result: 작은 예시를 손으로 계산하여(block_size=256: 길이 256→다음 단계에서 두 번째 블록 필요, 해당 조건이 특정 나머지에서 트리거됨) 검증할 수 있습니다.

Q16: 메모리가 부족하면 어떻게 되나요?

Situation: 추론 서비스는 압박 하에서도 붕괴되지 않고 디그레이드될 수 있어야 합니다.

Task: `can_allocate` / `can_append` 와 `preempt` 를 연결합니다.

Action: Prefill 시 `can_allocate` 가 실패하면, `schedule` 의 waiting 루프가 `break` 되어 요청이 계속 대기열에 머무릅니다; Decode 시 `can_append` 가 실패하면, `running` 꼬리에서 `preempt` 하고, `deallocate` 로 KV를 해제하여, 시퀀스를 `waiting` 에 되돌립니다(재계산 가능).

Result: 일부 요청의 지연을 희생하여 시스템 생존을 교환한다고 설명합니다; 그리고 "vLLM의 CPU로 swap 등의 메커니즘에 비해, 교습용 구현은 더 단순하다"고 솔직하게 말합니다.

소스 코드 앵커:

```

def schedule(self) -> tuple[list[Sequence], bool]:
    # prefill
    scheduled_seqs = []
    ...
    while self.waiting and num_seqs < self.max_num_seqs:
        seq = self.waiting[0]
        if num_batched_tokens + len(seq) > self.max_num_batched_tokens or not self.block_m
            break

```

```

while not self.block_manager.can_append(seq):
    if self.running:
        self.preempt(self.running.pop())
    else:
        self.preempt(seq)
        break

```

3. 스케줄링 및 배치 처리 (6문제)

Q17: 연속 배치 처리의 핵심 사상은 무엇인가요?

Situation: 정적 배치는 경계가 고정되어 있어, GPU가 배치 꼬리에서 유휴 상태가 되기 쉽습니다.

Task: 한 문장 + nano-vllm에서의 구현 위치.

Action: 매 라운드 `step` 마다 계산에 참여할 시퀀스 집합을 다시 선택하며, 완료된 것은 퇴장하고 새 요청이 진입합니다; `waiting` / `running` 이 동적 집합을 구현합니다.

Result: 「단계를 단위로 하는 동적 배치」임을 강조하며, 한 번의 forward로 모든 토큰을 계산하는 것이 아닙니다.

Q18: Prefill이 Decode보다 우선하는 이유는 무엇인가요? (본 저장소 schedule 기반)

Situation: 대기 중인 새 요청과 생성 중인 시퀀스가 동시에 있을 때, 누구를 먼저 서비스할지가 TTFT와 TPOT에 영향을 줍니다.

Task: `schedule` 이 먼저 `waiting`을 스캔하는 구현을 설명합니다.

Action: 코드에서 먼저 `prefill`의 `while self.waiting`을 실행하며, `scheduled_seqs`가 비어 있지 않으면 바로 `return`하여, 이번 라운드에서는 `decode` 구간을 실행하지 않습니다; 따라서 새로 도착하여 할당 가능한 Prefill은 동일 `step` 내의 Decode를 막습니다.

Result: 장단점을 논의할 수 있습니다: 대기 요청의 TTFT를 낮추는 데 유리; 이미 `running`에 있는 요청의 대기 시간을 증가시킬 수 있습니다(제품 SLA와 결합하여 설명 필요).

소스 코드 앵커:

```
while self.waiting and num_seqs < self.max_num_seqs:
    ...
    if scheduled_seqs:
        return scheduled_seqs, True
```

Q19: 선점 메커니즘은 어떻게 작동하나요?

Situation: Decode는 새 토큰을 위해 블록을 예약해야 하며, 여유 공간이 부족하면 양보해야 합니다.

Task: 어느 큐에서 빼앗는지, 빼앗은 후 상태가 어떻게 되는지 설명합니다.

Action: `can_append` 실패 시 `preempt(self.running.pop())`을 통해, 즉 `running` 큐 꼬리에서 시퀀스를 가져옵니다; `preempt` 는 상태를 `WAITING`으로 변경하고 `deallocate` 합니다.

Result: 이는 「꼬리 `running` 시퀀스를 희생」하는 전략입니다; FCFS와 비교하여 설명할 수 있습니다(구현 상으로는 `pop` 꼬리).

소스 코드 앵커:

```
def preempt(self, seq: Sequence):
    seq.status = SequenceStatus.WAITING
    self.block_manager.deallocate(seq)
    self.waiting.appendleft(seq)
```

Q20: schedule 메서드의 완전한 흐름은 무엇인가요?

Situation: 빈출 주제, 두 단계로 나누어 구술할 수 있어야 합니다.

Task: Prefill 구간 + Decode 구간 + 반환값의 의미.

Action:

1. `waiting`에서 시퀀스를 가져오려고 시도하며, `max_num_batched_tokens`와 `can_allocate`를 확인하고, 통과하면 `allocate`하여 `running`에 통합, `scheduled_seqs`로 수집합니다; 비어 있지 않으면 `(scheduled_seqs, True)`를 반환합니다.
2. 그렇지 않으면 `decode`로 진입: `running`에서 시퀀스를 가져와, `can_append`를 보장하고, 그렇지 않으면 선점; `may_append` 후 수집; 마지막으로 `running.extendleft(reversed(scheduled_seqs))`로 순서를 유지하며, `(scheduled_seqs, False)`를 반환합니다.

Result: 두 번째 반환값 `is_prefill`이 어떻게 `ModelRunner`에 전달되어 준비 경로에 영향을 미치는지 설명할 수 있습니다.

Q21: postprocess는 어떻게 시퀀스 종료를 판단하나요?

Situation: 종료 조건이 자원 해제에 영향을 줍니다.

Task: 조건과 부작용을 나열합니다.

Action: 각 `(seq, token_id)`에 대해: 먼저 `append_token`; 만약 `token_id == eos` (그리고 `ignore_eos`가 아님)이거나 `max_tokens`에 도달하면, `FINISHED`로 설정하고, `deallocate`하며, `running`에서 제거합니다.

Result: EOS와 길이 상한 두 가지 종료 유형을 설명합니다.

소스 코드 앵커: Q3 인용 `postprocess` 참조.

Q22: waiting과 running 큐의 관리 전략은 무엇인가요?

Situation: 큐의 의미론과 순서를 평가합니다.

Task: 새 요청이 어디로 들어가는지, Prefill 후 어디로 가는지, 선점 시 어디로 돌아가는지.

Action: `add`는 `waiting.append` 사용; Prefill 성공 후 `waiting.popleft`에서 `running.append`로 이동; 선점은 `waiting.appendleft`로 큐 헤드에 다시 삽입하여 재시도 우선권 부여.

Result: "running 순서가 decode 구간에서 `popleft` / `extendleft`에 의해 유지된다"는 직관을 설명할 수 있습니다.

4. 모델 및 계산 (8문제)

Q23: Qwen3의 모델 아키텍처는 무엇인가요?

Situation: 직무가 모델 측면을 다룬다면, 요약할 수 있어야 합니다.

Task: 모듈 파일로부터 답변: RMSNorm, RoPE, SwiGLU, GQA 등.

Action: `nanovllm/models/qwen3.py`와 결합: Decoder-only, 다층 Transformer Block; 어텐션 내 Q/K/V와 GQA 헤드 수 설정; FFN은 SwiGLU(gate/up/down) 사용; 위치 인코딩은 RoPE.

Result: "LLaMA 계열과 가까운 구체적인 변형은 `config.json` 기준"이라고 설명합니다.

Q24: GQA의 구현과 장점은 무엇인가요?

Situation: KV 메모리와 대역폭이 추론 병목입니다.

Task: 헤드 수 관계와 코드에서의 표현을 설명합니다.

Action: `num_attention_heads`가 `num_key_value_heads`보다 클 때, 여러 Q 그룹이 한 세트의 K/V를 공유합니다; `allocate_kv_cache`는 `num_kv_heads`로 블록 크기를 계산합니다; FlashAttention 측에서 GQA를 지원합니다.

Result: 장점: KV Cache가 더 작고, 메모리 접근이 적음; 대가는 MHA보다 표현력이 약간 낮음(아키텍처 설계로 균형).

Q25: RoPE는 위치 정보를 어떻게 인코딩하나요?

Situation: 셀프 어텐션 자체가 순열 불변이므로, 위치 의존성이 필요합니다.

Task: 회전이 Q/K에 작용하는 방식과 상대 위치 성질을 설명합니다.

Action: RoPE는 위치를 Q/K에 대한 회전 변환으로 의존시킵니다; 구현 상으로 `cos/sin` 테이블을 사전 계산하고 위치 인덱스에 따라 적용합니다(`rotary_embedding.py` 참조).

Result: 외삽 주제(NTK/YaRN)를 보충하면 가산점입니다.

Q26: SwiGLU의 계산 과정은 무엇인가요?

Situation: FFN 형태에 대한 빈출 암기 주제.

Task: 공식 수준의 직관 + 모듈명을 씁니다.

Action: 전형적인 형태 ($\text{FFN}(x) = (xW_{\text{gate}} \odot \sigma(xW_{\text{up}}))W_{\text{down}}$) (구체적으로 Qwen3 구현 기준); 활성화 함수는 보통 SiLU 사용.

Result: "단일 게이트 FFN 대비, SwiGLU는 한 개의 gate 경로가 추가로 정보 흐름을 제어한다"고 설명합니다.

Q27: RMSNorm과 LayerNorm의 차이점은 무엇인가요?

Situation: Norm 질문.

Task: 평균, 분산, 중심화 여부를 비교합니다.

Action: RMSNorm은 제곱 평균 제곱근으로 스케일링하고, 평균을 빼지 않습니다; 계산 비용이 더 저렴합니다; nano-vllm에서는 각 레이어의 `RMSNorm` 구현을 볼 수 있습니다.

Result: 한 문장으로: 더 가볍고, 대형 모델 학습/추론에 흔히 사용됨.

Q28: FlashAttention의 prefill과 decode 분기는 무엇인가요?

Situation: 동일 `Attention.forward` 내의 내부 분기.

Task: 해당 API와 컨텍스트 구성의 차이.

Action: Prefill: `flash_attn_varlen_func`, `cu_seqlens_q/k` 전달; 만약 prefix cache가 있고 `cu_seqlens_k > cu_seqlens_q`이면, `block_tables`를 설정하여 페이징 읽기 경로를 탐. Decode: `flash_attn_with_kvcache` + `cache_seqlens` + `block_tables`.

Result: varlen으로 가변 길이 패킹을 처리함을 강조; decode 단일 단계 단일 토큰의 KV cache 접근 패턴.

소스 코드 앵커: Q9 참조; 및 `prepare_prefill` / `prepare_decode`.

Q29: 샘플링 전략의 구현은 무엇인가요?

Situation: logits → 토큰의 '마지막 1km'.

Task: 온도, top-k/top-p, greedy의 관계.

Action: `layers/sampler.py` 읽기: `temperature` 스케일링; `top_p` / `top_k` 필터링; `torch.compile` 선택적 가속.

Result: `temperature=0`일 때 보통 greedy에 대응된다고 설명합니다(구현 세부사항은 코드 기준).

Q30: LMHead는 prefill 시 어떻게 마지막 토큰만 취하나요?

Situation: Prefill은 한 번에 여러 hidden을 계산하지만, 다음 단계 예측에는 마지막 위치만 필요합니다.

Task: `ParallelLMHead` 분기를 가리킵니다.

Action: `get_context().is_prefill`이 참일 때, `cu_seqlens_q[1:] - 1`을 사용하여 각 시퀀스의 마지막 위치 hidden을 취한 후, `F.linear`로 logits를 얻습니다.

Result: "패킹된 시퀀스" 표현과 일관되며, 전체 긴 시퀀스에 대해 무의미한 logits 계산을 피합니다(어텐션은 여전히 계산하지만, logits는 차원 축소).

소스 코드 앵커:

```
def forward(self, x: torch.Tensor):
    context = get_context()
    if context.is_prefill:
        last_indices = context.cu_seqlens_q[1:] - 1
        x = x[last_indices].contiguous()
        logits = F.linear(x, self.weight)
```

5. 시스템 최적화 (6문제)

Q31: CUDA Graph의 원리와 제한 사항은 무엇인가요?

Situation: Decode 런칭 오버헤드가 차지하는 비중이 높습니다.

Task: capture/replay와 적용 조건을 설명합니다.

Action: `capture_cudagraph`가 여러 `bs`에 대해 녹화합니다; `run_model`은 `prefill`이 아니고, `eager`가 아니며, `bs ≤ 512`일 때 그래프 `replay`를 선택합니다. 제한: 시퀀스 길이 변화가 크거나, 그래프 구조가 변하면 → `prefill`에 부적합; 동적 `shape`이 녹화 범위를 초과하면 `fallback`이 필요합니다.

Result: 「동형 그래프 + 자리 차지 버퍼 + copy」를 복기할 수 있습니다.

소스 코드 앵커:

```
def capture_cudagraph(self):
    ...
    for bs in reversed(self.graph_bs):
        graph = torch.cuda.CUDAGraph()
        ...
        with torch.cuda.graph(graph, self.graph_pool):
            outputs[:bs] = self.model(input_ids[:bs], positions[:bs])    # capture
```

Q32: Triton Kernel의 설계 사상은 무엇인가요?

Situation: K/V를 페이징 풀에 기록하려면 고처리량 `scatter`가 필요합니다.

Task: 병렬화 단위와 경계를 설명합니다.

Action: `store_kvcache_kernel`은 각 `program`이 하나의 토큰 위치에 대응합니다: `slot_mapping`을 읽고, -1이면 건너뛴다; `key/value` 벡터를 `k_cache` / `v_cache`의 펼쳐진 오프셋에 기록합니다.

Result: `slot_mapping`과 동일 길이를 강조하며, `prepare_prefill/decode`가 채워 넣습니다.

소스 코드 앵커:

```
@triton.jit
def store_kvcache_kernel(
    key_ptr,
    ...
):
    idx = tl.program_id(0)
    slot = tl.load(slot_mapping_ptr + idx)
    if slot == -1: return
    ...
    tl.store(k_cache_ptr + cache_offsets, key)
```

Q33: torch.compile은 어디에 사용되었나요?

Situation: PyTorch 2.x 컴파일 가속 핫스팟.

Task: 저장소 기준으로 나열합니다.

Action: 본 저장소에서 `@torch.compile`을 검색하면, 주로 다음에 나타납니다: `layers/sampler.py` (샘플링), `layers/activation.py` (SwiGLU 관련 활성화 함수 등), `layers/layernorm.py` (RMSNorm 등), `layers/rotary_embedding.py` (RoPE). 원칙: **파이썬 스케줄링 오버헤드 감소, 연산자 융합 용이**, 구체적인 이득은 PyTorch/CUDA 버전 및 `shape`에 따라 달라집니다.

Result: "샘플링, Norm, RoPE, 활성화 함수 등 핫스팟 서브 모듈을 커버한다"고 답변하고, **Profiler**로 실측한 후 이력서 결론에 쓰는 것을 권장합니다.

Q34: 텐서 병렬화의 구현 디테일은 무엇인가요?

Situation: 멀티 카드 가중치 샤딩과 통신.

Task: Column/Row 및 어텐션 출력 집계.

Action: `linear.py`에서 `ColumnParallel`은 먼저 국소 `matmul` 후 `gather`; `RowParallel`은 먼저 국소 연산 후 `all_reduce`; `init_process_group("nccl")`로 프로세스 그룹 구축.

Result: "레이어당 대략 두 번의 `all_reduce`" 수준을 셀 수 있습니다(구체적인 것은 모듈 구현에 따라 다름).

Q35: SharedMemory 통신 메커니즘은 무엇인가요?

Situation: TP>1일 때 비주 rank는 파이썬 스케줄링을 직접 실행하지 않으며, 패킷을 받아 실행해야 합니다.

Task: rank0이 공유 메모리에 쓰고 Event를 사용하는 것을 설명합니다.

Action: `write_shm` 이 `pickle`로 `[method_name, args]`를 직렬화합니다; 자식 프로세스 `read_shm` 이 `event`를 대기합니다; `call`을 호출하여 `run/exit`으로 분배합니다.

Result: 이것이 단일 프로세스 멀티스레드가 아닌 멀티프로세스 TP 구동 IPC임을 설명합니다.

소스 코드 앵커:

```
def write_shm(self, method_name, *args):
    assert self.world_size > 1 and self.rank == 0
    data = pickle.dumps([method_name, *args])
    ...
    for event in self.event:
        event.set()
```

Q36: pin_memory의 역할은 무엇인가요?

Situation: CPU→GPU 복사와 파이프라인.

Task: page-locked memory를 설명합니다.

Action: `prepare_prefill` 등에서 `pin_memory=True` + `non_blocking=True`: 비동기 DMA, 계산과 중첩되어 단계 별 동기 대기를 줄입니다.

Result: 「H2D 블로킹 감소, 처리량 향상」이라고 답변하면 충분합니다.

소스 코드 앵커:

```
input_ids = torch.tensor(input_ids, dtype=torch.int64, pin_memory=True).cuda(non_block
positions = torch.tensor(positions, dtype=torch.int64, pin_memory=True).cuda(non_block
```

6. 개방형 문제 (6문제)

Q37: 더 많은 모델을 지원하려면 어떻게 해야 하나요?

Situation: 산업적 요구는 흔히 다중 아키텍처입니다.

Task: 확장 영역을 지적합니다: 설정, 가중치, 레이어 구현.

Action: `models/xxx.py`를 새로 추가하여 `ForCausalLM`을 정의합니다; `load_model` / `weight_loader`가 명명 규칙에 적응합니다; `ModelRunner`에서 해당 클래스를 인스턴스화합니다; `Attention`과 KV shape, RoPE 파라미터가 일관되도록 보장합니다.

Result: 「엔진 계층은 재사용, 모델 계층은 교체」임을 강조합니다.

Q38: 스트리밍 출력을 추가하려면 어떻게 해야 하나요?

Situation: 제품 경험에서 흔한 요구사항.

Task: 「`step`마다 토큰을 산출」하는 것으로부터 인터페이스를 유도합니다.

Action: `generate` 루프에서 매 단계 현재 증가된 토큰 또는 텍스트를 `yield`합니다; 스레드 안전성과 HTTP chunked 응답에 주의합니다; 스케줄링 로직은 변경하지 않을 수 있습니다.

Result: nano-vllm은 기본적으로 `step` 외부에 콜백을 추가하여 구현할 수 있다고 설명합니다.

Q39: 온라인 서비스(API Server)를 구현하려면 어떻게 해야 하나요?

Situation: 배포 형태가 스크립트에서 서비스로 전환됩니다.

Task: 프로세스를 분할합니다: 요청 접수, 스케줄링 스레드, GPU 워커.

Action: FastAPI/Flask로 요청 접수 → 큐에 `add_request` 투입 → 백그라운드 스레드가 `step` 루프; 멀티 카드 시 rank0 통일 스케줄링 유지; 헬스 체크 및 동시성 제한 추가.

Result: 배치와 지연의 트레이드오프 및 OpenAI 호환 레이어를 가산점으로 지적할 수 있습니다.

Q40: 양자화 추론을 지원하려면 어떻게 해야 하나요?

Situation: nano-vllm은 기본적으로 FP16/BF16입니다.

Task: 어떤 연산자 경로를 변경해야 하는지 설명합니다.

Action: 가중치를 INT8/FP8로 양자화; `Linear` 를 양자화 커널로 대체; KV 양자화는 `store_kvcache` 와 어텐션 API 변경 필요; 교정 및 스케일링 전략(GPTQ/AWQ)은 오프라인 완료.

Result: "작업량은 연산자와 정밀도 검증에 있다"고 솔직하게 말합니다.

Q41: 추측적 디코딩을 구현하려면 어떻게 해야 하나요?

Situation: decode를 가속하는 최첨단 방향.

Task: 초안 모델 + 검증 단계를 설명합니다.

Action: 스케줄러에 '초안으로 k 토큰 생성 → 대형 모델이 한 번에 검증'을 삽입해야 합니다; `Sampler` , KV 재사용 전략과 밀접하게 관련됩니다; nano-vllm은 대대적인 개조가 필요합니다.

Result: 검증 실패 시 어떻게 토큰과 KV를 롤백하는지 알고 있음을 보여줍니다.

Q42: 대규모 배포 시 어떤 점을 고려해야 하나요?

Situation: 단일 머신에서 클러스터로.

Task: SLO, 내결함성, 탄력성, 모니터링.

Action:

- 용량: 카드 당 동시 요청 수, KV 메모리, PD 분리 필요 여부;
- 신뢰성: 프로세스 재시작, 요청 타임아웃, 재시도;
- 관측: TTFT, TPOT, 큐 길이, GPU SM/메모리;
- 버전: 모델과 tokenizer 버전 관리.

Result: "nano-vllm에서 배운 것은 핵심 알고리즘", "운영 투입 시 운영 및 SRE를 보완해야 한다"고 설명합니다.

부록: 시험 준비 제안

1. 우선적으로 완전히 이해할 것: `scheduler.py` , `block_manager.py` , `model_runner.py` , `attention.py` .
2. 대조 실험: `enforce_eager=True` 로 CUDA Graph를 비활성화하고, Decode 처리량을 비교합니다.
3. 정직하게 표현할 것: 학습 프로젝트와 독창적 엔지니어링의 경계를, 노트/실험/커밋으로 깊이를 증명합니다.